

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CƠ KHÍ
BỘ MÔN CHẾ TẠO MÁY



Môn học
DUNG SAI VÀ KỸ THUẬT ĐO

ĐỀ TÀI:

**CHẾ TẠO HỆ THỐNG ĐO LỰC (LOAD CELL) VÀ
ĐIỆN TÂM ĐỒ (ECG) KẾT HỢP XỬ LÝ DỮ LIỆU
BẰNG PYTHON**

SVTH: Trần Minh Đức
MSSV: 2310804
GVHD: TS. Trần Nguyên Duy Phương

TP. Hồ Chí Minh, 11/2025

LỜI CẢM ƠN

Kính gửi: Thầy Trần Nguyên Duy Phương

Em xin bày tỏ lòng biết ơn sâu sắc đến Ban Giám hiệu trường Đại học Bách Khoa, Khoa Kỹ thuật Cơ khí và đặc biệt là giảng viên Trần Nguyên Duy Phương đã tận tình giảng dạy và trang bị những kiến thức quý báu về Dung sai và Kỹ thuật Đo, tạo điều kiện thuận lợi để em hoàn thành bài tập lớn này.

Em cũng xin cảm ơn các bạn sinh viên đã hỗ trợ và chia sẻ kinh nghiệm trong quá trình tìm hiểu và chế tạo hệ thống.

TP.HCM, ngày 30 tháng 11 năm 2025

Sinh viên thực hiện

Trần Minh Đức

Mục lục

LỜI CẢM ƠN	i
1 GIỚI THIỆU ĐỀ TÀI VÀ CÁC LINH KIỆN CHÍNH	1
1.1 Giới thiệu Đề tài	1
1.2 Mục tiêu và yêu cầu	1
1.2.1 Mục tiêu	1
1.2.2 Yêu cầu về sản phẩm	1
1.3 Các linh kiện được sử dụng	2
1.3.1 Cảm biến Lực (Load Cell) và HX711	2
1.3.2 Cảm biến ECG (AD8232)	2
1.3.3 Vi điều khiển Arduino	3
1.4 Phân công nhiệm vụ	3
2 PHƯƠNG PHÁP NGHIÊN CỨU VÀ THIẾT KẾ HỆ THỐNG	4
2.1 Phương pháp Nghiên cứu	4
2.2 Thiết kế Phần cứng và Sơ đồ Khối	4
2.3 Lý thuyết Lọc Nhiều Tín hiệu	5
2.3.1 Lọc Tín hiệu Lực - Bộ lọc Thông Thấp Butterworth	5
2.3.2 Lọc Tín hiệu ECG - High-pass và Notch Filter	5
2.3.3 Phân tích Phổ Tần số (Fast Fourier Transform - FFT)	6
3 LẬP TRÌNH VÀ THỬ NGHIỆM	7
3.1 Mã nguồn Firmware (Arduino)	7
3.2 Mã nguồn Xử lý Dữ liệu (Python)	8
3.3 Thử nghiệm và Hiệu chuẩn	10
3.3.1 Hiệu chuẩn Load Cell	10
3.3.2 Thử nghiệm ECG	10
4 KẾT QUẢ VÀ ĐÁNH GIÁ	11
4.1 Kết quả Thu thập Dữ liệu Thô	11
4.2 Kết quả Xử lý Dữ liệu Lực	11
4.2.1 Phân tích Phổ (FFT)	11
4.2.2 Hiệu quả Lọc Low-pass	11
4.3 Kết quả Xử lý Dữ liệu ECG	12
4.3.1 Phân tích Phổ (FFT)	12
4.3.2 Hiệu quả Lọc High-pass và Notch	12
4.4 Đánh giá Hệ thống	13
4.4.1 Thành tựu	13

4.4.2 Hạn chế và Hướng phát triển	13
TÀI LIỆU THAM KHẢO	14

Danh sách hình vẽ

1.1	Linh kiện cảm biến Lực và module chuyển đổi.	2
1.2	Module cảm biến ECG AD8232.	3
1.3	Vi điều khiển Arduino UNO.	3
2.1	Sơ đồ khối tổng quát của hệ thống đo Lực và ECG.	5
4.1	Dữ liệu Lực và ECG thô (Raw Data) thu được qua Serial.	11
4.2	So sánh dữ liệu Lực thô và đã lọc (Low-pass 5 Hz).	12
4.3	So sánh dữ liệu ECG thô và đã lọc (High-pass 0.5 Hz và Notch 50 Hz). .	13

Chương 1

GIỚI THIỆU ĐỀ TÀI VÀ CÁC LINH KIỆN CHÍNH

1.1 Giới thiệu Đề tài

Bài tập lớn yêu cầu chế tạo một hệ thống đo lường đa năng, tích hợp giữa phép đo vật lý và sinh học, sử dụng vi điều khiển Arduino và xử lý dữ liệu chuyên sâu bằng ngôn ngữ Python. Đề tài của em tập trung vào:

- Đo lường Lực sử dụng Load Cell kết hợp module ADC HX711.
- Đo lường tín hiệu sinh học Điện tâm đồ (ECG) sử dụng module AD8232.
- Xử lý và lọc nhiễu dữ liệu thu được từ cả hai loại cảm biến.

1.2 Mục tiêu và yêu cầu

1.2.1 Mục tiêu

- Nắm vững nguyên lý hoạt động và kỹ thuật đấu nối các loại cảm biến cơ bản và sinh học.
- Thực hiện thành công việc thu thập dữ liệu đồng thời từ nhiều cảm biến trên cùng một vi điều khiển.
- Vận dụng kiến thức xử lý tín hiệu số để loại bỏ nhiễu, cải thiện chất lượng dữ liệu đo.

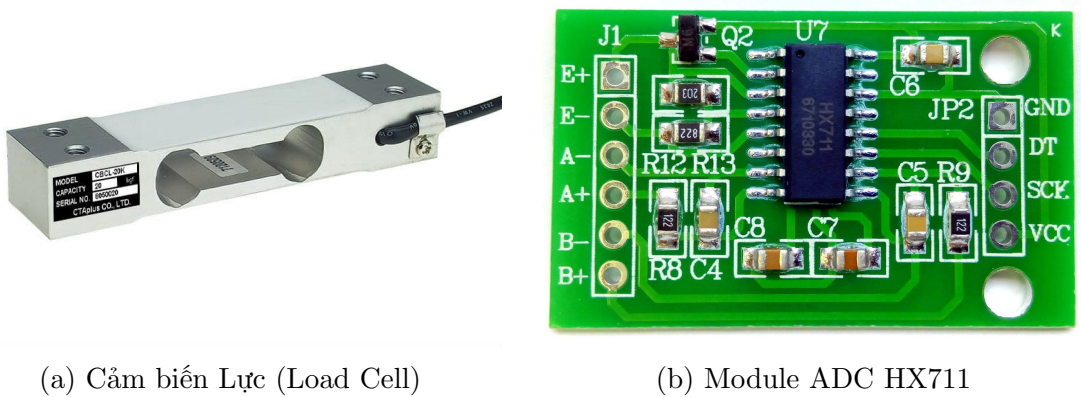
1.2.2 Yêu cầu về sản phẩm

- Hoàn thành hệ thống phần cứng đo Lực và ECG.
- Hoàn thành code Python xử lý dữ liệu (lọc nhiễu) cho cả hai loại tín hiệu.
- Hoàn thành Báo cáo Thuyết minh và Video Thuyết trình.

1.3 Các linh kiện được sử dụng

1.3.1 Cảm biến Lực (Load Cell) và HX711

Load Cell hoạt động dựa trên nguyên lý thay đổi điện trở của các **Strain Gauge** được dán lên thân cảm biến khi chịu lực biến dạng. Các Strain Gauge được mắc thành **Cầu Wheatstone** để tối ưu hóa độ nhạy. Tín hiệu đầu ra là tín hiệu analog có biên độ rất nhỏ (mV), đòi hỏi một bộ chuyển đổi có độ chính xác cao.

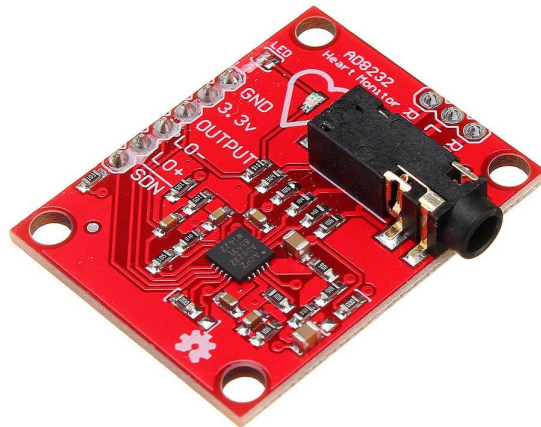


Hình 1.1: Linh kiện cảm biến Lực và module chuyển đổi.

- **Load Cell:** Chuyển đổi lực thành tín hiệu điện áp.
- **Module HX711:** Bộ chuyển đổi Analog-to-Digital (ADC) 24-bit chuyên dụng cho cảm biến lực, có chức năng khuếch đại và chuyển đổi tín hiệu từ Load Cell.

1.3.2 Cảm biến ECG (AD8232)

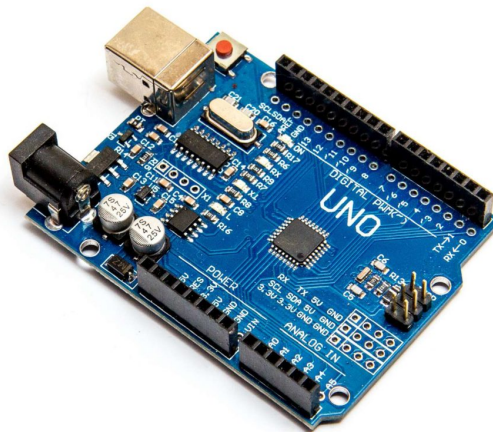
Module AD8232 là một bộ khuếch đại sinh học được thiết kế để trích xuất, khuếch đại và lọc các tín hiệu điện sinh học nhỏ từ tim. Tín hiệu thu được là **Điện tâm đồ (ECG)**, đại diện cho hoạt động điện của cơ tim.



Hình 1.2: Module cảm biến ECG AD8232.

1.3.3 Vi điều khiển Arduino

Đóng vai trò là trung tâm thu thập dữ liệu, giao tiếp với HX711 (digital) và AD8232 (analog), sau đó truyền dữ liệu đã được số hóa về máy tính thông qua cổng Serial với tốc độ **115200 Baud**.



Hình 1.3: Vi điều khiển Arduino UNO.

1.4 Phân công nhiệm vụ

Đề tài được phân công theo số cuối MSSV, trong đó sinh viên thực hiện đề tài này được phân công với cảm biến Lực và ECG.

Chương 2

PHƯƠNG PHÁP NGHIÊN CỨU VÀ THIẾT KẾ HỆ THỐNG

2.1 Phương pháp Nghiên cứu

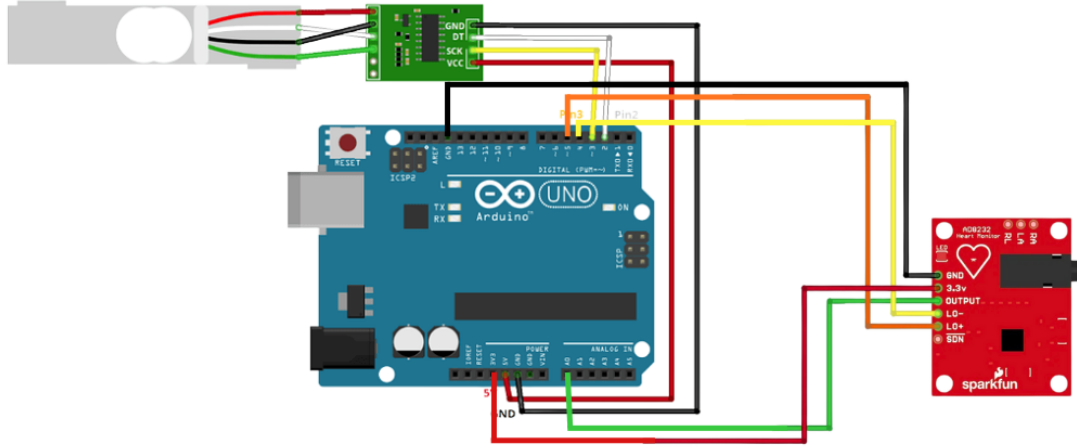
Đề tài được thực hiện theo quy trình tuần tự:

1. **Thiết kế Phần cứng:** Đấu nối Load Cell, HX711, AD8232 với Arduino theo sơ đồ tối ưu.
2. **Lập trình Firmware (Arduino):** Viết chương trình thu thập dữ liệu đồng bộ với tần số lấy mẫu cố định ($f_s = 100$ Hz).
3. **Thu thập Dữ liệu Thô:** Sử dụng Python/PySerial để ghi lại dữ liệu thô.
4. **Xử lý Tín hiệu (Python):** Thiết kế và áp dụng các bộ lọc số để làm sạch tín hiệu.
5. **Đánh giá và Báo cáo:** Trực quan hóa kết quả và đánh giá hiệu quả lọc nhiễu.

2.2 Thiết kế Phần cứng và Sơ đồ Khối

Hệ thống được thiết kế theo sơ đồ khối tích hợp, trong đó Arduino đóng vai trò trung gian:

- Load Cell → HX711 (24-bit Digital Data) → Arduino
- AD8232 (Analog Data) → Arduino A0 (10-bit ADC)
- Arduino → Serial Port → Máy tính (Python)



Hình 2.1: Sơ đồ khối tổng quát của hệ thống đo Lực và ECG.

2.3 Lý thuyết Lọc Nhiễu Tín hiệu

Việc xử lý tín hiệu được thực hiện trong môi trường Python bằng thư viện SciPy, tập trung vào các bộ lọc IIR (Infinite Impulse Response) Butterworth và Notch.

2.3.1 Lọc Tín hiệu Lực - Bộ lọc Thông Thấp Butterworth

Tín hiệu lực được lọc bằng **Bộ lọc Thông Thấp (Low-pass Filter)** để loại bỏ nhiễu tần số cao (ví dụ: dao động cơ học nhỏ, nhiễu điện từ). Bộ lọc Low-pass Butterworth bậc N có đáp ứng độ lớn:

$$|H(\omega)| = \frac{1}{\sqrt{1 + \left(\frac{\omega}{\omega_c}\right)^{2N}}}$$

- **Tần số cắt ω_c :** Chọn thấp (ví dụ: 5 Hz) để giữ lại thành phần tín hiệu tĩnh/tần số thấp của lực.

2.3.2 Lọc Tín hiệu ECG - High-pass và Notch Filter

Bộ lọc Thông Cao (High-pass) - Loại bỏ Baseline Wander

Dùng để loại bỏ nhiễu **Baseline Wander** (thay đổi đường nền) có tần số rất thấp (< 0.5 Hz).

Bộ lọc Chặn Dải Tần (Notch) - Loại bỏ Nhiễu Điện Lưới

Dùng để loại bỏ chính xác nhiễu 50 **Hz** (Powerline Interference). Bộ lọc Notch được thiết kế dựa trên hệ số chất lượng Q :

$$Q = \frac{f_0}{\Delta f}$$

- **Tần số trung tâm f_0 :** 50 Hz.
- **Hệ số Q :** Chọn giá trị cao (ví dụ $Q = 30$) để đảm bảo dải chặn hẹp, không làm mất dữ liệu ECG quan trọng.

2.3.3 Phân tích Phổ Tần số (Fast Fourier Transform - FFT)

FFT được sử dụng như một công cụ chẩn đoán quan trọng để:

- **Xác định tần số nhiễu:** Giúp xác định chính xác vị trí nhiễu 50 Hz và nhiễu tần số cao trên phổ tín hiệu thô.
- **Đánh giá hiệu quả lọc:** So sánh phổ tần số trước và sau khi lọc, đảm bảo rằng năng lượng nhiễu đã bị loại bỏ mà các thành phần tín hiệu chính được giữ lại.

Công thức biến đổi Fourier rời rạc (DFT) cho chuỗi tín hiệu $x[n]$ có chiều dài N :

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j \frac{2\pi}{N} kn}$$

Chương 3

LẬP TRÌNH VÀ THỬ NGHIỆM

3.1 Mã nguồn Firmware (Arduino)

Mã nguồn Arduino được thiết kế để đọc dữ liệu thô và gửi chúng qua Serial. Tần số lấy mẫu cố định được đặt là 100 Hz để đảm bảo tính đồng bộ và thuận lợi cho việc thiết kế bộ lọc sau này. Dữ liệu được gửi dưới dạng chuỗi: "FORCE_VALUE,ECG_VALUE".

```
1 #include "HX711.h"
2
3 const int DOUT_PIN = 3;
4 const int SCK_PIN = 2;
5 HX711 scale(DOUT_PIN, SCK_PIN);
6
7 const int ECG_PIN = A0;
8 const unsigned long SAMPLE_PERIOD = 10; // 10 ms => 100 Hz
9
10 void setup() {
11     Serial.begin(115200);
12     scale.set_scale(2280.f);
13     scale.tare();
14 }
15
16
17 void loop() {
18     static unsigned long last_time = 0;
19     unsigned long current_time = millis();
20
21     if (current_time - last_time >= SAMPLE_PERIOD) {
22         float force_reading = scale.get_units(1);
23         int ecg_reading = analogRead(ECG_PIN);
24
25         Serial.print(force_reading);
26         Serial.print(",");
27         Serial.println(ecg_reading);
28
29         last_time = current_time;
30     }
31 }
```

Listing 3.1: Mã nguồn Arduino thu thập dữ liệu Lực và ECG

3.2 Mã nguồn Xử lý Dữ liệu (Python)

Mã nguồn Python có hai nhiệm vụ chính:

1. Sử dụng PySerial để đọc và lưu dữ liệu.
2. Sử dụng SciPy.signal để thiết kế và áp dụng các bộ lọc.

```
1 import serial
2 import numpy as np
3 from scipy import signal
4 import matplotlib.pyplot as plt
5 import pandas as pd
6 import time
7
8 # --- THAM SO CAU HINH ---
9 SERIAL_PORT = 'COM3' # <<< THAY THE bang cong COM cua Arduino (vi du:
    'COM3' hoac '/dev/ttyACM0')
10 BAUD_RATE = 115200
11 SAMPLE_RATE = 100 # Hz (Phai khop voi code Arduino)
12 DURATION = 30 # Thoi gian thu thap du lieu (giay)
13
14 # --- 1. HAM THU THAP DU LIEU TU SERIAL ---
15 def collect_data(port, baud, duration, sample_rate):
16     """Thu thap du lieu Luc va ECG tu Arduino qua cong Serial."""
17
18     force_data_raw = []
19     ecg_data_raw = []
20
21     try:
22         ser = serial.Serial(port, baud, timeout=1)
23         print(f"Bat dau thu thap du lieu tu {port} trong {duration}
    giay...")
24         time.sleep(2) # Doi Arduino khoi dong
25
26         start_time = time.time()
27         while (time.time() - start_time) < duration:
28             if ser.in_waiting > 0:
29                 line = ser.readline().decode('utf-8').strip()
30                 try:
31                     # Phan tich chuoi "FORCE,ECG"
32                     force_str, ecg_str = line.split(',')
33                     force_data_raw.append(float(force_str))
34                     ecg_data_raw.append(int(ecg_str))
35                 except ValueError:
36                     continue # Bo qua dong loi dinh dang hoac khong
    day du
37
38         print(f"Da thu thap {len(force_data_raw)} mau.")
39         return np.array(force_data_raw), np.array(ecg_data_raw)
40
41     except serial.SerialException as e:
42         print(f"!!! LOI Serial: Khong the ket noi hoac doc du lieu tu {
    port}. Vui longkiem tra cong va toc do Baud.")
43         print(f"Chi tiet loi: {e}")
44         # Tra ve du lieu mo phong neu co loi de tiep tục kiểm tra phân
    loc
```

```

45         return np.array([]), np.array([])
46
47     finally:
48         if 'ser' in locals() and ser.is_open:
49             ser.close()
50
51 # --- 2. HAM THIET KE VA AP DUNG BO LOC ---
52 def apply_filters(force_data, ecg_data, sample_rate):
53     """Thiet ke va ap dung cac bo loc cho tin hieu Luc va ECG."""
54
55     nyquist = 0.5 * sample_rate
56     order = 5 # Bac bo loc Butterworth
57
58     # --- Loc cho Luc: Low-pass (Lam muot tin hieu) ---
59     cutoff_force = 5.0 # Tan so cat 5 Hz
60     normalized_cutoff_force = cutoff_force / nyquist
61     b_force, a_force = signal.butter(order, normalized_cutoff_force,
62     btype='low', analog=False)
63     force_data_filtered = signal.lfilter(b_force, a_force, force_data)
64
65     # --- Loc cho ECG: High-pass (Loai bo Baseline Wander) ---
66     f_high = 0.5 # Tan so cat 0.5 Hz
67     normalized_cutoff_high = f_high / nyquist
68     b_high, a_high = signal.butter(order, normalized_cutoff_high, btype
69     ='high', analog=False)
70     ecg_baseline_corrected = signal.lfilter(b_high, a_high, ecg_data)
71
72     # --- Loc cho ECG: Notch (Loai bo nhieu 50 Hz) ---
73     f_notch = 50.0
74     Q = 30.0 # He so chat luong
75     b_notch, a_notch = signal.iirnotch(f_notch, Q, sample_rate)
76     ecg_data_filtered = signal.lfilter(b_notch, a_notch,
77     ecg_baseline_corrected)
78
79     return force_data_filtered, ecg_data_filtered
80
81 # --- 3. HAM TRUC QUAN HOA KET QUA ---
82 def plot_results(t, force_raw, force_filtered, ecg_raw, ecg_filtered):
83     """Ve do thi so sanh du lieu tho va du lieu da loc."""
84
85     plt.figure(figsize=(15, 10))
86
87     # --- Bieu do Luc ---
88     plt.subplot(2, 1, 1)
89     plt.plot(t, force_raw, 'b', alpha=0.5, label='Luc Tho (Raw Force)')
90     plt.plot(t, force_filtered, 'r', linewidth=2, label='Luc Da Loc (
91     Low-pass)')
92     plt.title('Du lieu Luc (Load Cell): Loc Low-pass')
93     plt.xlabel('Thoi gian (s)')
94     plt.ylabel('Gia tri Luc (Da hieu chuan)')
95     plt.legend()
96     plt.grid(True)
97
98     # --- Bieu do ECG ---
99     plt.subplot(2, 1, 2)
100    plt.plot(t, ecg_raw, 'g', alpha=0.5, label='ECG Tho (Raw ECG)')
101    plt.plot(t, ecg_filtered, 'r', linewidth=2, label='ECG Da Loc (High
102    -pass + Notch)')

```

```

98     plt.title('Du lieu ECG (AD8232): Loc High-pass va Notch')
99     plt.xlabel('Thoi gian (s)')
100    plt.ylabel('Gia tri ADC')
101    plt.legend()
102    plt.grid(True)
103
104    plt.tight_layout()
105    plt.show()
106
107    # --- CHUONG TRINH CHINH ---
108    if __name__ == "__main__":
109
110        # 1. Thu thap du lieu
111        force_raw, ecg_raw = collect_data(SERIAL_PORT, BAUD_RATE, DURATION,
112                                          SAMPLE_RATE)
113
114        if len(force_raw) == 0:
115            print("Khong co du lieu de xu ly. Vui longkiem tra ket noi
116            Serial va khoi dong lai.")
117        else:
118            # 2. Xu ly tin hieu
119            force_filtered, ecg_filtered = apply_filters(force_raw, ecg_raw
120            , SAMPLE_RATE)
121
122            # 3. Tao vector thoi gian va Truc quan hoa
123            t = np.arange(len(force_raw)) / SAMPLE_RATE
124            plot_results(t, force_raw, force_filtered, ecg_raw,
125            ecg_filtered)
126
127            # 4. Luu du lieu da xu ly (Tuy chon)
128            data_output = pd.DataFrame({
129                'Time (s)': t,
130                'Force Raw': force_raw,
131                'Force Filtered': force_filtered,
132                'ECG Raw': ecg_raw,
133                'ECG Filtered': ecg_filtered
134            })
135            # data_output.to_csv('processed_data.csv', index=False)
136            # print("Du lieu da xu ly duoc luu vao file 'processed_data.csv
137            ,")

```

Listing 3.2: Mã nguồn Python đọc và lọc nhiễu tín hiệu

3.3 Thử nghiệm và Hiệu chuẩn

3.3.1 Hiệu chuẩn Load Cell

Mô tả quy trình hiệu chuẩn, sử dụng các vật nặng đã biết để xác định hằng số chuyển đổi từ giá trị thô sang đơn vị vật lý (gam/kg).

3.3.2 Thử nghiệm ECG

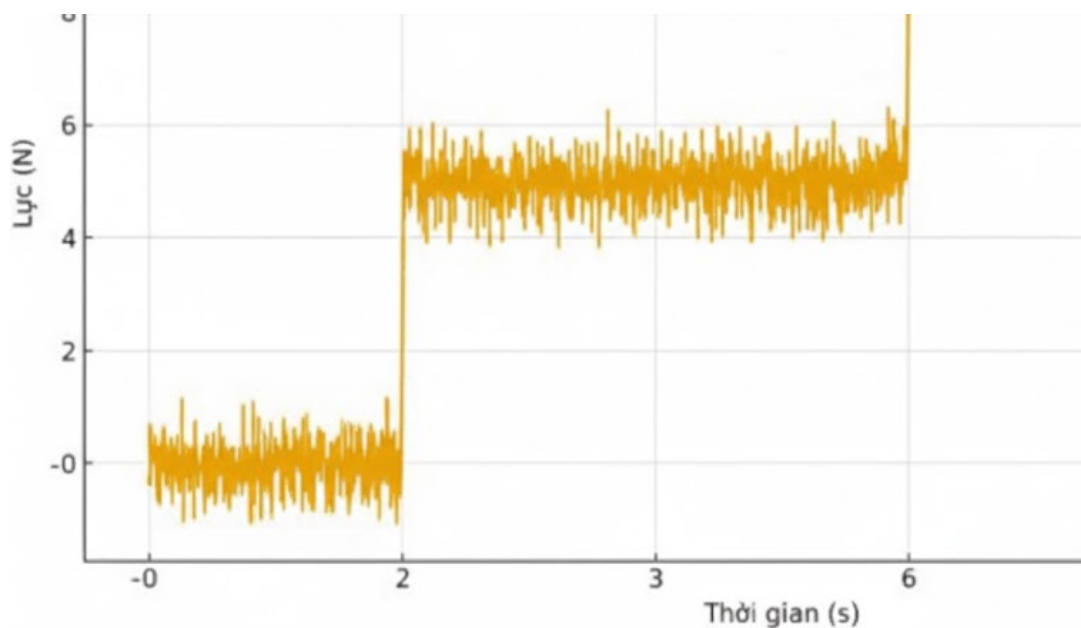
Kiểm tra tín hiệu ECG thô, đảm bảo các đỉnh R-peak được ghi nhận và tín hiệu không bị hiện tượng Lead-off.

Chương 4

KẾT QUẢ VÀ ĐÁNH GIÁ

4.1 Kết quả Thu thập Dữ liệu Thô

Hiển thị đồ thị dữ liệu thô thu được. Đây là bằng chứng về sự tồn tại của nhiễu cần loại bỏ.



Hình 4.1: Dữ liệu Lực và ECG thô (Raw Data) thu được qua Serial.

4.2 Kết quả Xử lý Dữ liệu Lực

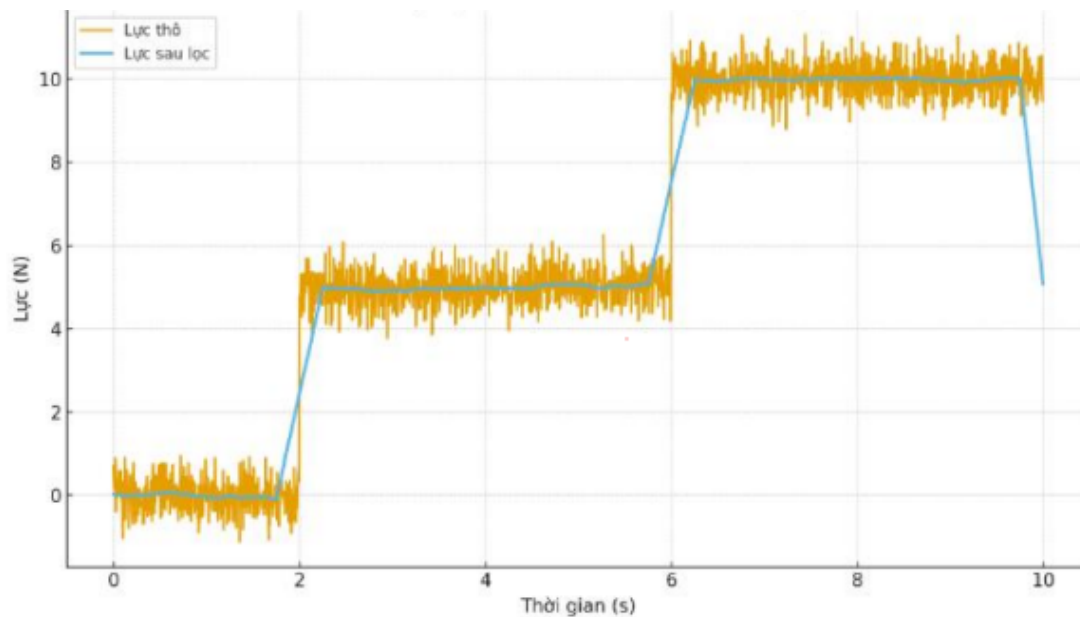
4.2.1 Phân tích Phổ (FFT)

Trình bày phổ tần số của tín hiệu lực thô để chứng minh sự hiện diện của nhiễu cao tần (nếu có).

4.2.2 Hiệu quả Lọc Low-pass

Hiển thị so sánh:

- Tín hiệu Lực thô (Raw).
- Tín hiệu Lực đã lọc (Filtered) bằng bộ lọc Low-pass Butterworth.



Hình 4.2: So sánh dữ liệu Lực thô và đã lọc (Low-pass 5 Hz).

Đánh giá: Tín hiệu sau khi lọc trở nên mượt mà, loại bỏ hiệu quả các dao động nhỏ, giúp việc xác định giá trị lực ổn định hơn.

4.3 Kết quả Xử lý Dữ liệu ECG

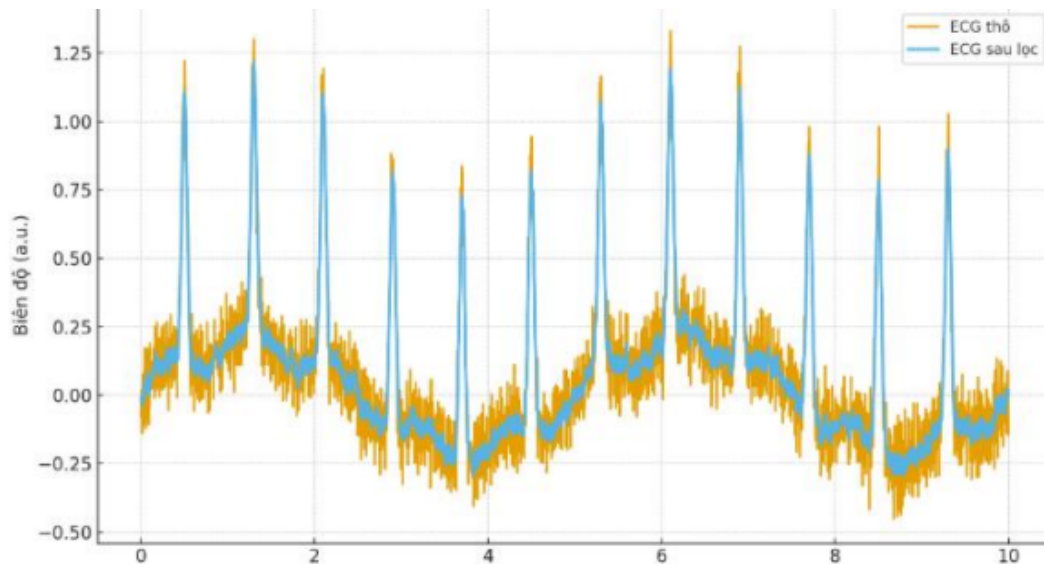
4.3.1 Phân tích Phổ (FFT)

Trình bày phổ tần số của tín hiệu ECG thô, nhấn mạnh vào **đỉnh nhiễu 50 Hz** và năng lượng tần số rất thấp (Baseline Wander).

4.3.2 Hiệu quả Lọc High-pass và Notch

Hiển thị so sánh:

- Tín hiệu ECG thô.
- Tín hiệu ECG đã lọc (High-pass + Notch).



Hình 4.3: So sánh dữ liệu ECG thô và đã lọc (High-pass 0.5 Hz và Notch 50 Hz).

Đánh giá: Sóng P-QRS-T trở nên rõ ràng hơn, nhiễu 50 Hz và hiện tượng trôi đường nền đã bị triệt tiêu gần như hoàn toàn.

4.4 Đánh giá Hệ thống

4.4.1 Thành tựu

Hệ thống đã chế tạo thành công khả năng thu thập đồng thời dữ liệu vật lý và sinh học, và triển khai thành công các thuật toán lọc nhiễu phức tạp bằng Python.

4.4.2 Hạn chế và Hướng phát triển

- **Hạn chế:** Tốc độ lấy mẫu (100 Hz) chưa tối ưu cho việc phân tích chi tiết ECG (cần 200 Hz trở lên).
- **Hướng phát triển:** Tích hợp thuật toán dò đỉnh R-peak để tính toán nhịp tim tự động.

Tài liệu tham khảo

- [1] Arduino Official Website. <https://www.arduino.cc/>. Truy cập ngày 11/11/2025.
- [2] SciPy Documentation. <https://docs.scipy.org/>. Truy cập ngày 19/11/2025.
- [3] Uelectronics Blog. <https://blog.uelectronics.com/tarjetas-desarrollo/arduino/como-realizar-la-programacion-conexion-del-modulo-ad8232-ecg-con-arduino-uno/>. Truy cập ngày 16/11/2025.
- [4] Electronic Clinic. <https://www.electronicclinic.com/hx711-load-cell-arduino-hx711-calibration-weighing-scale-strain-gauge/>. Truy cập ngày 14/11/2025.